

Proposal builder as a PowerPoint add-in: plan

Oct 6, 2026 · @Sharya

Recommendation

Yes, build template management as a PowerPoint add-in, but keep one Python engine behind it. The add-in should replace the FastAPI *web console* (the screens authors use), not the engine or the need for a small server.

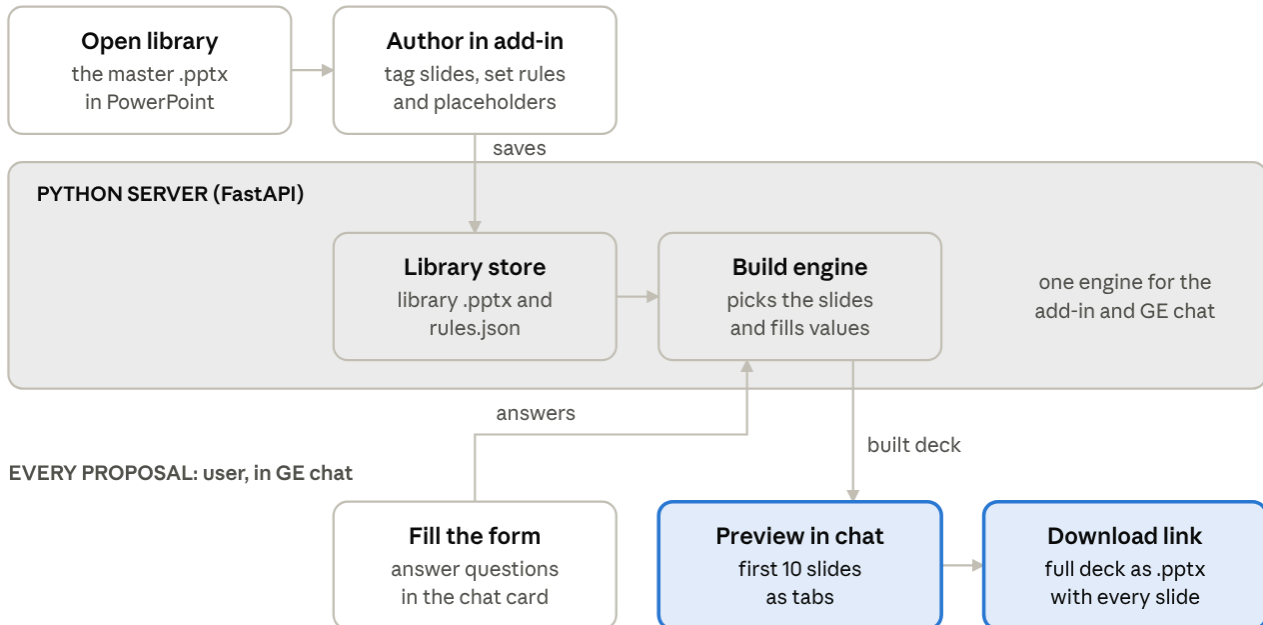
- **Authors work where the library already lives.** They open `library.pptx` in PowerPoint and a task pane gives them the Slides, Questions, Rules and Values screens beside the real slides.
- **The preview problem goes away for authors and users.** The built deck opens as a real PowerPoint window with every value filled. That is what v3p1 spent the Graph spike trying to get (7 s per preview, a confidential copy briefly in SharePoint, a beta purge API). In the add-in it is free.
- **A server still has to exist**, but it shrinks to a headless engine API plus shared storage. Three reasons: rules and libraries must be shared between authors, Gemini Enterprise chat needs to build decks server-side anyway, and porting the engine to JavaScript would leave two engines that must stay identical.
- **Do not rebuild the deck with PowerPoint's JavaScript calls.** The OOXML engine (`assemble.py` , `placeholders.py`) already handles split runs, tables and repair-dialog traps. Office.js has no find-and-replace for slide text and cannot edit SmartArt or charts, so the add-in should receive a finished `.pptx` from the engine and open it.

The first gate is cheap and decides everything: confirm the firm lets a custom add-in run in its PowerPoint at all (Phase 0 below).

□ How it flows

Authors set rules in PowerPoint; a Python server builds every deck for GE chat

SET UP ONCE: author, in the PowerPoint add-in



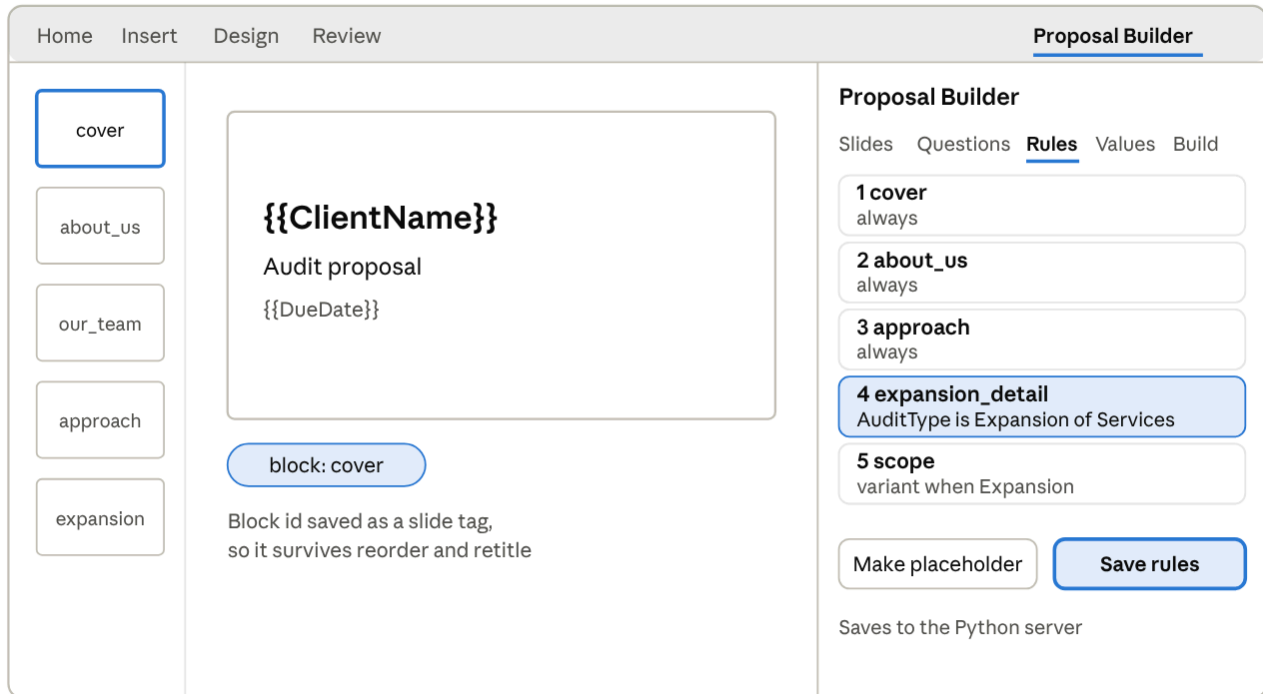
end-to-end flow · 3 author steps, 4 user steps

Authors set up a library once in PowerPoint, and the add-in saves it to the Python server. After that, the same server builds every proposal from answers given in Gemini Enterprise chat: the first 10 slides show as tabs, and the full deck comes as a download link.

Mock-up screens

These are illustrations for discussion, not designs of the final screens. Slide names and conditions come from the demo library; the client name and the 42-slide count are examples.

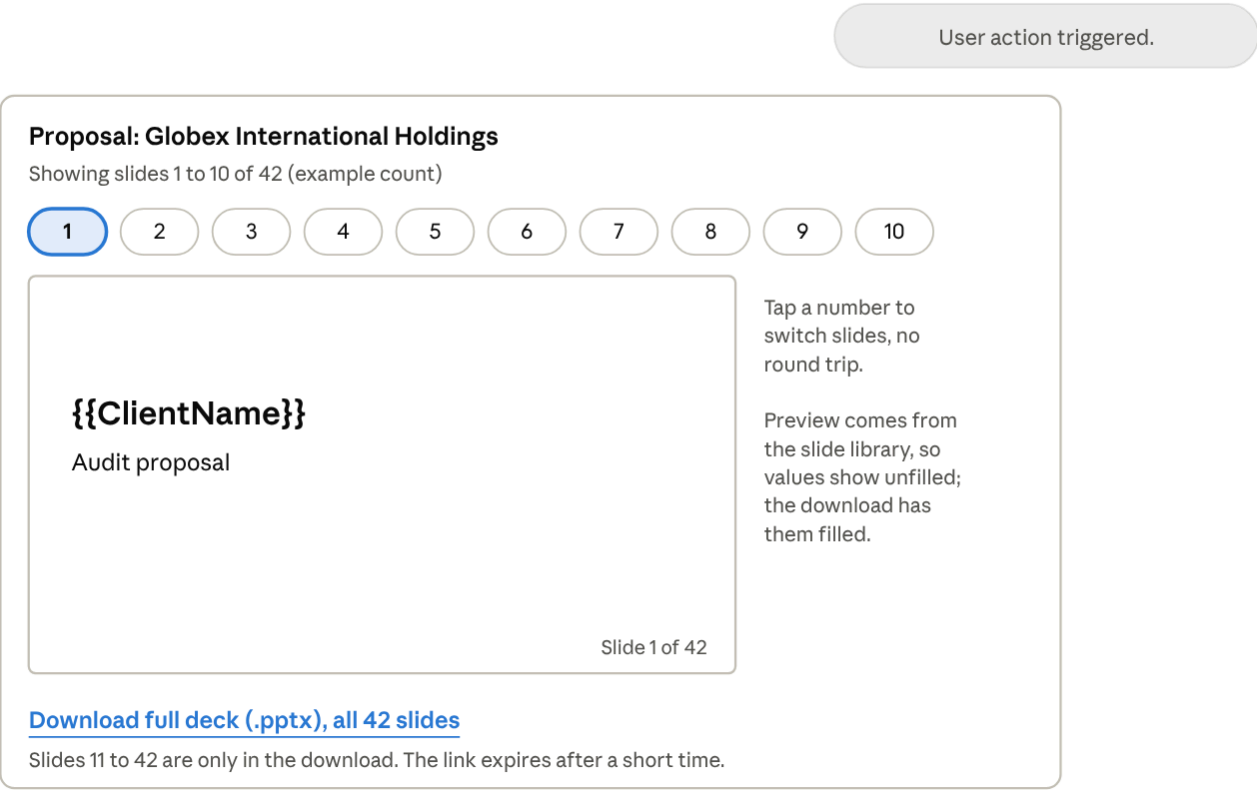
In PowerPoint, the author sets rules in a side pane next to the real slides



mock-up · PowerPoint add-in, Rules screen

The author works on the real library slides. The pane on the right carries the existing Rules, Values and Build screens, and the highlighted row is a conditional slide.

In GE chat, the user sees the first 10 slides as tabs and downloads the full deck



mock-up · GE chat card, 10 tabs and a download link

The user picks a slide by its number tab, as the tabs-deck probe proved works in GE. Every slide, including those past 10, comes in the download.

How it works today

`ppt_gen_v3p1` is `ppt_gen_v3` plus a Microsoft Graph preview backend. The engine is about 6,400 lines of pure Python with no Office dependency (the build path itself, rules to finished zip, is about 2,000); `app.py` is 1,200 lines of thin FastAPI endpoints over it.

Piece	Where	What it does
Template folder	<code>templates/<id>/</code>	<code>library.pptx</code> , <code>library.pdf</code> , <code>blocks.json</code> (slide to block id), <code>rules.json</code> , <code>data_sources.json</code> . <code>rules.json</code> on disk is the source of truth.
Slide identity	<code>engine/library.py</code>	<code>{{block:id}}</code> marker, else <code>blocks.json</code> , else slugified title, else position. A slide retitled in the same edit reads as one gone and one new.

Piece	Where	What it does
Rules	<code>engine/rules.py</code>	<code>baseline</code> spine, conditional blocks with <code>when</code> (<code>eq</code> , <code>in</code> , <code>exists</code> , <code>all</code> , <code>any</code> , <code>not</code>) and <code>insert_after</code> , and <code>variant</code> to swap content in place.
Values	<code>engine/bindings.py</code> , <code>placeholders.py</code>	<code>{{Placeholder}}</code> bound to an answer field, a literal or a data source, with date formats. Runs split by PowerPoint are stitched before replacing.
Build	<code>engine/assemble.py</code>	Copies the selected slide parts out of the library zip, renumbers ids, fixes content types and rels. Deterministic.
Preview	<code>engine/render.py</code> , <code>graph.py</code>	Pages cut from the library PDF (fast, shows <code>{{ClientName}}</code> unfilled); Graph conversion (filled, about 7 s, needs SharePoint).
Checks	<code>verify.py</code> , <code>suggest.py</code> , <code>tools/</code>	Compare against Templafy decks; suggest rules from real decks.

The README lists what is not yet true: no authentication, local filesystem storage, one writer per library, builds never cleaned up. Any move off localhost has to fix those whatever the front end is.

What moves into the add-in, and what stays on a server

Every author screen moves into the add-in. Storage, the build engine and the Templafy checks stay server-side, behind a smaller API.

Today (FastAPI console)	In the add-in	Still on a server	Why
Slides: import library, mark blocks	Task pane on the open <code>library.pptx</code> ; block ids written as slide tags	Stores the library file	Tags survive reorder and retitle, fixing the "one gone, one new" case. The pane sends the open file with <code>getFileAsync</code> , so no manual upload.

Today (FastAPI console)	In the add-in	Still on a server	Why
Make library PDF (needs PowerPoint COM today)	Possibly: export slide images from the open deck, if the firm's build supports it	Stores the page images	Removes the one step that needed desktop COM. Needs checking on the firm's PowerPoint version.
Questions: upload answer sets	Same screen, in the pane	Field catalogue (<code>payloads.py</code>)	Catalogue is shared between authors and with GE chat.
Rules: order and conditions	Pane editor; library order can follow the PowerPoint thumbnail order	<code>rules.json</code> , validation, backups	Dragging slides in PowerPoint is the natural reorder; the narrow pane suits the existing "move to #" control.
Values: bind placeholders	Select text on the slide, click "Make placeholder", bind it in the pane	Bindings in <code>rules.json</code>	Authors no longer type <code>{{Name}}</code> by hand.
Build a deck	Form in the pane, then the built deck opens as a new PowerPoint window	<code>assemble.build</code>	One engine for add-in and GE chat; deterministic output kept.
Preview (library PDF, Graph)	Not needed: the real deck is open, filled, at full fidelity	Only for GE chat	Drops the Graph app registration, SharePoint round trip and beta purge for this path.
Check against Templafy, Suggest rules	Buttons in the pane	<code>verify.py</code> , <code>suggest.py</code>	Batch jobs over many decks; no reason to port.

The existing `web/app.js` UI is plain HTML and JavaScript, so a good share of it can be carried into the task pane rather than rewritten.

PowerPoint JavaScript API: what it can and cannot do

The API covers everything the author pane needs, and nothing it would need to replace the engine. Requirement sets PowerPointApi 1.1 to 1.10 run on Windows, Mac and the web; the iPad stops at 1.1, and perpetual Office (LTSC) only from 1.5 (Office 2024).

Need	API	Set	Fit
Open the built deck in a new window	<code>PowerPoint.createPresentation(base64)</code>	1.1	Good: this is the preview.
Insert built slides into the open deck instead	<code>insertSlidesFromBase64</code>	1.2	Good, optional.
Stamp a block id on each slide	slide tags	1.3	Good. The engine must learn to read tags (<code>library.py</code>) and keep stripping them on output (<code>assemble.py</code> already strips customer data).
Read the current slide order	slides collection	1.3	Good for driving the baseline order.
Insert a placeholder at the selection	<code>getSelectedTextRange</code>	1.6	Good.
Send the open library to the server	<code>Office.context.document.getFileAsync</code> (compressed)	common API	Good; sliced upload.

Need	API	Set	Fit
Replace {{Name}} across all slides	none: no search/replace in PowerPoint	n/a	Poor. Setting a whole text range loses run formatting. Keep substitution in the OOXML engine.
Edit SmartArt or chart text	not exposed	n/a	Not possible. The engine treats these as opaque parts, which is fine.
Tables	1.8 and 1.9	1.8+	Possible but unneeded; the engine already handles tables.

Deployment. An add-in is a manifest plus a static web app over HTTPS (Azure Static Web Apps or App Service). The firm's Microsoft 365 admin publishes it to users through Integrated Apps; developers sideload it. Sign-in uses the user's Microsoft 365 account through an Entra app registration with delegated permissions, which also gives the engine API the authentication it lacks today.

Phased build plan

Five phases, about 9 to 13 weeks for one developer who knows the codebase. Efforts are my estimate from the code size, not measured. Each phase ends at a gate; a failed gate changes the plan before more is built.

Phase	What gets built	Gate to pass	Rough effort
0. Prove add-ins are allowed	A hello-world task pane sideloaded into the firm's PowerPoint (desktop and web); it reads slide tags and calls createPresentation	It loads under the firm's policy, and the admin agrees to publish through Integrated Apps	2 to 3 days, plus admin lead time

Phase	What gets built	Gate to pass	Rough effort
1. Headless engine API	Split <code>app.py</code> into a JSON-only API; add Entra sign-in and an author role; move storage behind one interface (SharePoint library or Azure Blob); clean up <code>data/builds/</code>	<code>check_api.py</code> , <code>check_fixtures.py</code> , <code>verify_decks.py</code> still pass; every call needs a signed-in user	2 to 3 weeks
2. Engine: tags as identity	<code>library.py</code> reads slide tags as the first identity source; marker, <code>blocks.json</code> and title stay as fallbacks	Fixture library re-imported after reorder and retitle keeps every rule	3 to 5 days
3. Author pane	Slides, Questions, Rules, Values screens in the task pane, reusing <code>web/app.js</code> ; "Make placeholder" from the selection; library uploaded from the open file	An author takes <code>fixtures/master.pptx</code> to three <code>MATCH</code> results without leaving PowerPoint	3 to 4 weeks
4. Build pane and retire the console	Build form in the pane; built deck opens filled in a new window; Check against Templafy button	A non-author builds a deck and sees filled values with no Graph call	1 to 2 weeks

The FastAPI web console can stay up read-only through Phase 4 as a fallback, then be removed. The Graph preview backend stops being needed for authors and users; keep it only if GE chat wants filled previews (next section).

Later: building the deck in Gemini Enterprise chat

The `ge_chat_spike` probes already prove a preview can be shown in GE chat with no storage; the download and the form are still unproven. Building it after the add-in costs little, because the GE agent imports the same Python engine and reads the same rules.

What the probes showed (office GE, A2UI v0.9, 15 September 2026, from `PROBE.md`):

Finding	Consequence for the build
<code>data:</code> URI images render on v0.9	Slide images travel inside the message; no public or signed image URL needed.

Finding	Consequence for the build
Tabs, chips bound to the Image URL, and thumbnail-opens-Modal all navigate with no round trip	Use a thumbnail grid plus Modal for big decks, Tabs or chips for small ones. A Slider cannot pick stored slides.
Each A2UI part is dropped silently above about 1 MB	Keep each part under 800 KB; send leaf components first and the root last (<code>split-60</code> worked, 60 thumbnails, 1.66 MB).
One agent event fails with 400 "exceeded limit" between 3.4 MB and 6.3 MB	60 photo-heavy slides at 1200 px (16 MB) do not fit in one event. Round 6 (spreading a card over several events) is still pending.

Not yet proven in the files I read: T3 (does a markdown link to a signed `.pptx` URL download from GE) and T4 (does a realistic form return values intact, the date format, the nested-path rule). Both decide the design and should run before building.

Proposed flow.

1. The agent builds an A2UI v0.9 form from the same field catalogue the add-in uses, with flat paths only; dates are converted to `YYYYMMDD`.
2. On Build, it calls `assemble.build` in-process (pure Python, runs in Agent Engine) against the library and `rules.json` from shared storage.
3. It shows the first 10 slides as Tabs, one slide image per tab (the tabs-deck probe rendered 9 tabs at about 400 KB), split into parts under 800 KB. These show `{{ClientName}}` unfilled, exactly as v3 does today, and the card says so.
4. It gives the full deck, every slide including those past 10, as a short-lived signed `.pptx` link. T3 must prove that link downloads from GE first.

Storage decision this forces. The agent runs on Google Cloud and the add-in in Microsoft 365. If libraries live in SharePoint, the agent needs Graph access from Google Cloud; if they live in Cloud Storage, the add-in's API needs Google credentials. Publishing a read-only copy of each library and its rules to Cloud Storage on every rules save is the simplest bridge.

Risks and open questions

- ☐ **Will the firm allow a custom add-in?** Admins can block sideloading and must publish centrally. This is Phase 0 and the go/no-go.
- ☐ **Which PowerPoint do authors have?** LTSC 2021 and older lack the newer sets; the iPad has almost none. Authors need Microsoft 365 PowerPoint on Windows, Mac or the web.

- ☐ **Where do libraries and rules live?** SharePoint (fits Microsoft 365 sign-in) or Azure Blob, plus a Cloud Storage copy for GE. Confidentiality rules in `CLAUDE.md` section 6 apply to every option.
- ☐ **Does a stakeholder expect "no server at all"?** Possible only by porting the engine to TypeScript and storing everything in SharePoint. I advise against it: two engines to keep identical, and GE chat still needs the Python one.
- ☐ **Slide image export from the add-in.** If the firm's PowerPoint cannot export slide images, `make_library_pdf.py` stays a desktop step for the GE preview.
- ☐ **GE download and form (T3, T4)** are untested; Round 6 of the probe is pending.
 - **Graph spike open items** (fidelity with SmartArt, charts and the corporate font; second-stage recycle bin) matter only if Graph previews are kept for GE. The add-in path does not need them.

Sources

- Code and docs read on your device: `Templfy/proposal_builder/ppt_gen_v3p1` (`CLAUDE.md`, `README.md`, `engine/`, `docs/graph-findings.md`, `docs/graph-access-request.md`) and `Templfy/ge_chat_spike` (`ge-chat-spike.md`, `PROBE.md`).
- [PowerPoint JavaScript API requirement sets](#), Microsoft Learn.